

FATEC POMPEIA

Introdução à Computação em Nuvem – ISW055 – SISTEMAS INTELIGENTES

P1

Relatório bimestral de atividades

Avaliação individual – 2026.2

Lucas Tetsuya Hiratuca

Prof. Allan Lincoln Rodrigues Siriani

Pompeia, 07/10/2026

Sumário

- 1 Introdução
- 2 Metodologia
- 3 Quadro de entregas
- 4 Atividades realizadas
 - 4.1 Atividade 1 – Agenda telefônica em Flask
 - 4.2 Atividade 2 – Catálogo de filmes – Tom Hanks
 - 4.3 Atividade 3 – Desacoplando o login – microserviço de autenticação
 - 4.4 Atividade 4 – Controle de acesso por papel – RBAC
 - 4.5 Atividade 5 – Logs e auditoria
 - 4.6 Atividade 6 – Upload e perfil de usuário
- 5 Considerações finais
- 6 Declaração de autoria

1 Introdução

A disciplina ISW055 – Introdução à Computação em Nuvem, ministrada pelo Prof. Allan Lincoln Rodrigues Siriani na FATEC Pompeia no semestre letivo 2026.2, tem como propósito central proporcionar aos alunos a vivência prática e conceitual na construção, containerização e orquestração de aplicações modernas para ambientes em nuvem.

O projeto condutor ao longo deste primeiro bimestre foi o desenvolvimento e a evolução do Catálogo de Filmes de Tom Hanks. Partindo de uma arquitetura inicial monolítica, a solução progrediu de forma incremental para uma infraestrutura desacoplada em microsserviços containerizados com Docker. Foram incorporados requisitos reais de engenharia de software em nuvem: autenticação segura via JSON Web Tokens (JWT), controle de acesso baseado em papéis (RBAC), auditoria assíncrona de eventos com Redis Streams e persistência de mídia em object storage compatível com S3 (Garage S3).

Este relatório bimestral documenta minuciosamente as seis atividades propostas e entregues, detalhando as decisões de arquitetura adotadas, as dificuldades técnicas superadas, as evidências de entrega com carimbo temporal no Git e os resultados operacionais comprovados do sistema.

2 Metodologia

Todas as atividades foram desenvolvidas com controle de versão via Git em repositórios públicos no GitHub. As datas e os horários de entrega foram rigorosamente extraídos do histórico de commits (git log) e conferidos diretamente na interface web do GitHub, garantindo rastreabilidade e imutabilidade das comprovações.

O ambiente de execução e teste foi estruturado com Docker Desktop e Docker Compose, padronizando os contêineres entre os ambientes de desenvolvimento e produção. O deploy final em nuvem é gerenciado via Portainer no servidor disponibilizado pelo orientador (lucas-hiratuca-isw055.lapps.studio), exposto por túnel seguro Cloudflare. As imagens oficiais foram publicadas no Docker Hub no namespace lucashiratuca/.

Vale destacar, que o desenvolvimento da aplicação foi efetuado com auxílio do Antigravity, da Google, junto com os conhecimentos de arquitetura e boas práticas passadas na aula pelo professor, sendo aplicadas no desenvolvimento de cada entrega das features.

3 Quadro de entregas

Nº	Atividade	Descrição	Data planejada	Data realizada	Situação
1	Agenda telefônica em Flask	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em MariaDB.	07/08/2026	07/08/2026 20:03	entregue
2	Catálogo de filmes – Tom Hanks	Consumo da API TMDb, persistência em MariaDB e segregação por usuário.	20/08/2026	20/08/2026 20:54	entregue
3	Desacoplando o login	Login, cadastro e esqueci-minha-senha num serviço à	28/08/2026	27/08/2026 13:00	entregue

parte na rede interna
do Docker.

4	Controle de acesso – RBAC	O campo role passa a decidir permissões reais no backend (403 para usuário comum).	04/09/2026	01/09/2026 00:48	entregue
5	Logs e auditoria	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.	25/09/2026	08/09/2026 12:57	entregue
6	Upload e perfil de usuário	Página de perfil com avatar no Garage S3; só a referência fica no banco relacional.	02/10/2026	26/09/2026 15:11	entregue

4 Atividades realizadas

4.1 Atividade 1 – Agenda telefônica em Flask

Descrição	Nivelamento em sala: sistema monolítico Flask + Jinja com persistência em banco de dados MariaDB na nuvem.
Data planejada	07/08/2026
Data realizada	07/08/2026 20:03 - 17/08/2026 20:27
Situação	entregue
Evidência	GitHub – commits 28230dc, 738ce73 e b1345e4 + README com menção ao professor + print do sistema
Link	https://github.com/LucasHiratuca/aula_01_cloud

O que foi feito. Durante a aula inicial de nivelamento, foi desenvolvida uma aplicação web em Python utilizando o framework Flask e o motor de templates Jinja2. A aplicação implementa um CRUD de contatos telefônicos conectado a uma instância remota do MariaDB hospedada no IP 35.226.64.52.

Como decisão técnica, a camada de acesso a dados foi isolada utilizando a biblioteca PyMySQL, configurando codificação utf8mb4 e tratamento de erros de conexão. Posteriormente, foi aplicado saneamento de credenciais via variáveis de ambiente com arquivo .gitignore (commit fa5f58d).

Este projeto foi simples e rápido, com um desenvolvimento da API feita pelo ChatGPT, no notebook da faculdade inicialmente, e finalização da API junto com criação de um Front-End básico na minha máquina pessoal (Funcionalidade bônus).

Vale destacar que os commits não seguem boas práticas por não reportar o que foi feito, mas em suma, o “b” e “a” abordam a demanda pedida em sala de aula, enquanto o “commit2” integra a aplicação usando Front-End e uma conexão com MariaDB (Usando .env).

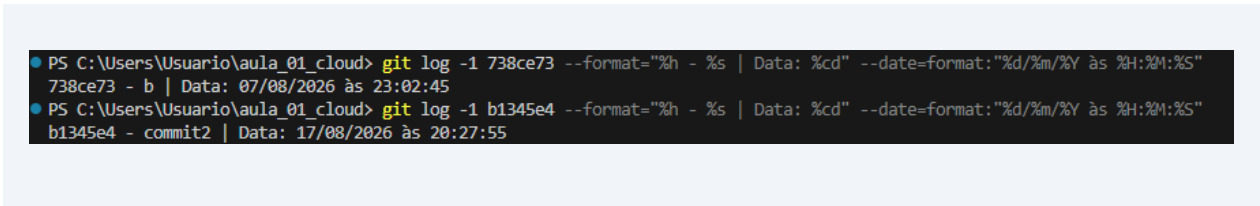


Figura 1: Atividade 1 – Evidência do commit

The image shows a dark-themed web form titled "Novo Cliente" (New Client). It features three input fields: "Nome completo" (Full Name) containing "Allan", "Telefone" (Phone) containing "23812981291892", and "Valor (R\$)" (Value in R\$) containing "900". A blue button with a paper plane icon and the text "Cadastrar Cliente" (Register Client) is at the bottom.

Novo Cliente

Nome completo

Allan

Telefone

23812981291892

Valor (R\$)

900

Cadastrar Cliente

Figura 2: Atividade 1 – Montagem do body para o POST no Front-End

 Clientes Cadastrados	
5 clientes encontrados	
NOME lucas	TELEFONE 1287121212
ENDEREÇO Rua das Flores, 9100	VALOR R\$ 93.00
NOME Trollman	TELEFONE 1213243314
ENDEREÇO Avenida Paulista, 2970	VALOR R\$ 54.00
NOME mt troll	TELEFONE 14134314
ENDEREÇO Rua do Sol, 2360	VALOR R\$ 41123423.00
NOME Cyrene	TELEFONE 093
ENDEREÇO Praça da Matriz, 1908	VALOR R\$ 33550338.00
NOME Allan	TELEFONE 23812981291892
ENDEREÇO Rua XV de Novembro, 5461	VALOR R\$ 900.00

Figura 3: Atividade 1 – Resultado do GET no Front-End

Dificuldades e como foram resolvidas. A principal dificuldade inicial consistiu na latência e na autenticação segura contra a base de dados remota do laboratório. O problema foi sanado com o correto ajuste da porta MySQL e a parametrização das variáveis no ambiente local. Um problema e confusão que aconteceu na parte de testes, foi tentar rodar essa aplicação com LiveServer, o que acabava em erro.

4.2 Atividade 2 – Catálogo de filmes – Tom Hanks

Descrição	Consumo da API TMDB, persistência em MariaDB e segregação por usuário com sistema de favoritos.
Data planejada	20/08/2026
Data realizada	20/08/2026 20:54
Situação	entregue
Evidência	GitHub – commit 9a57391 + README com menção ao professor + print do catálogo
Link	https://github.com/LucasHiratuca/Entregada-Atividade/commit/9a57391

O que foi feito. Criação da aplicação principal do projeto utilizando Node.js, Express e EJS. A aplicação consome a API pública do The Movie Database (TMDB), filtrando e exibindo a filmografia completa do ator Tom Hanks com posters, títulos originais, sinopses e notas de avaliação.

Foi construído o sistema de autenticação local com express-session e senhas criptografadas com bcryptjs. Adicionalmente, foi implementada a funcionalidade de filmes favoritos vinculada ao ID do usuário autenticado no MariaDB, segregando a lista de favoritos por sessão ativa.

```
5b8bfbe - Add GitHub link for Professor Siriani in README | 20/08/2026 às 21:02:18
9a57391 - Meu projeto do Tom Hanks | 20/08/2026 às 20:54:04
```

Figura 4: Atividade 2 – Evidência do commit

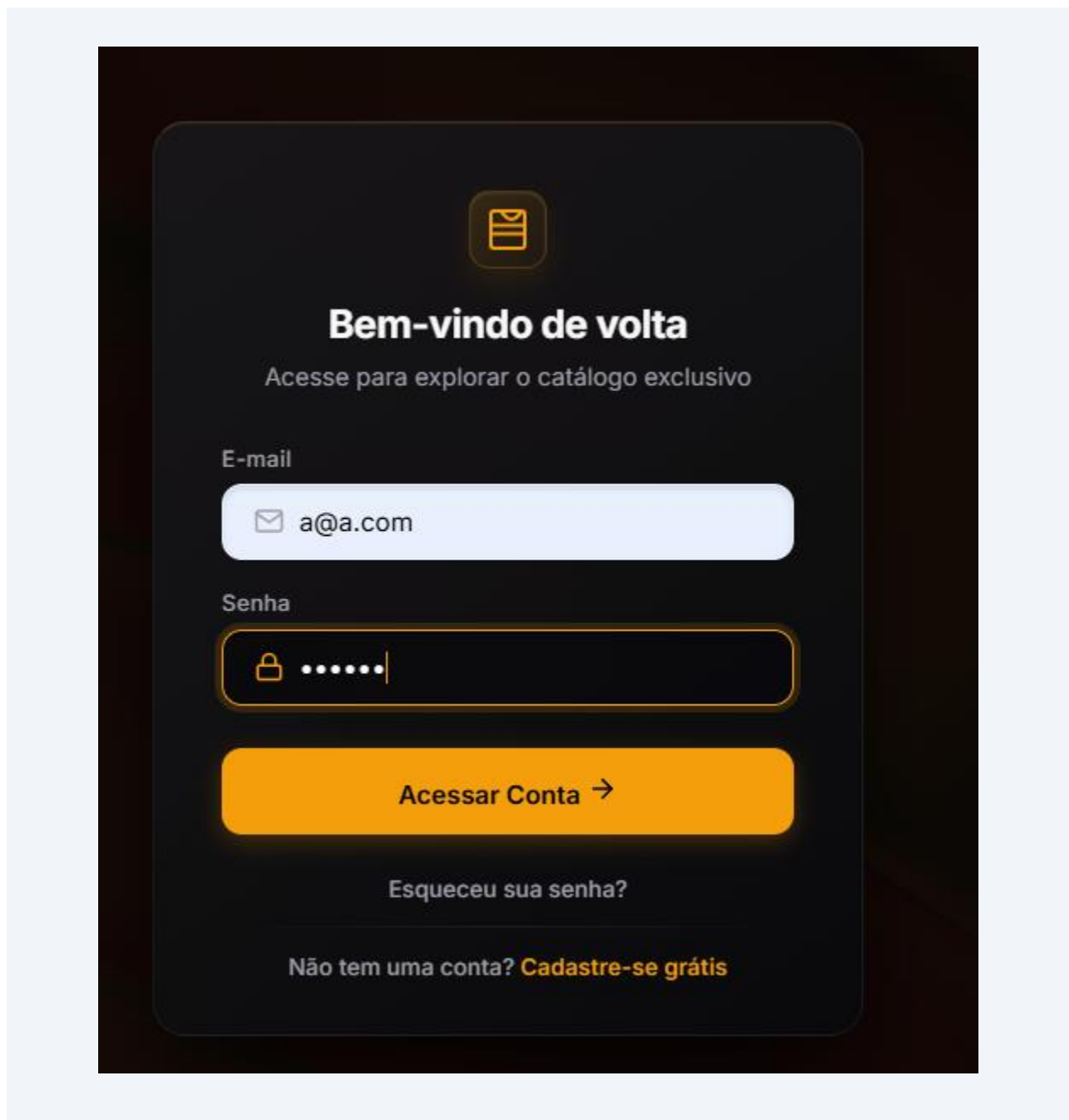


Figura 5: Atividade 2 – Tela de login



Bem-vindo de volta

Acesse para explorar o catálogo exclusivo

❗ E-mail ou senha inválidos.

E-mail

✉ seu@email.com

Senha

🔒 ••••••••

Acessar Conta →

[Esqueceu sua senha?](#)

Não tem uma conta? [Cadastre-se grátis](#)

Figura 6: Atividade 2 – Validação na tela de login

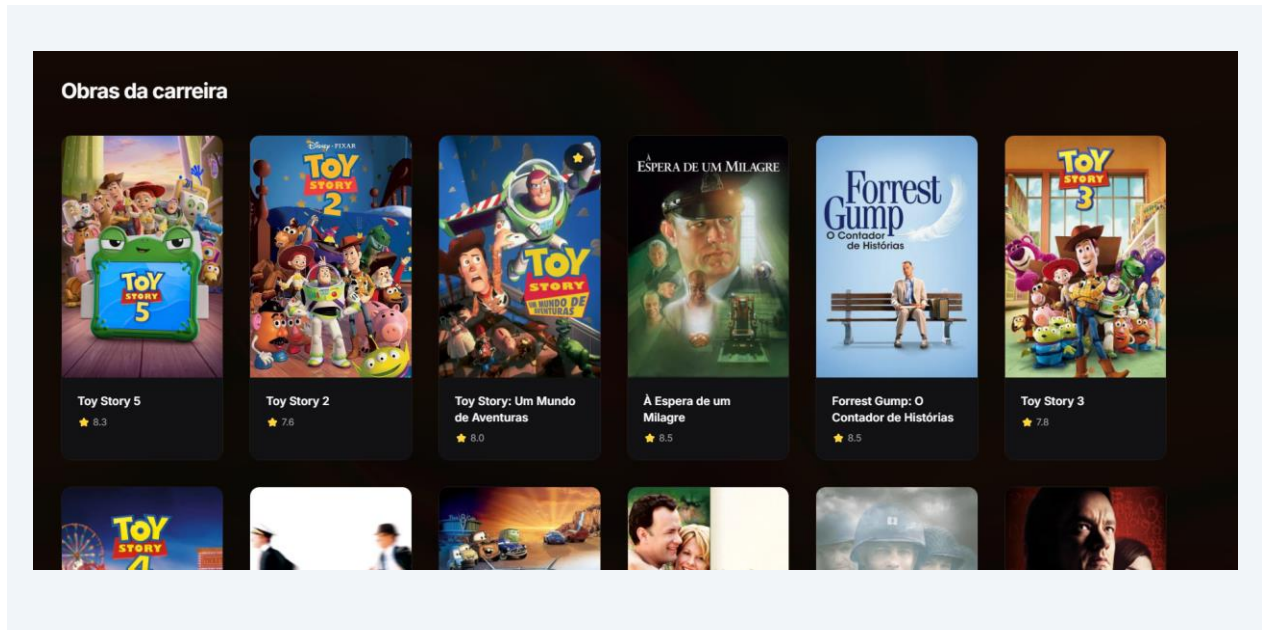


Figura 7: Atividade 2 – API pegando os filmes

DESCRIBE usuarios | Insira uma expressão SQL para filtrar os resultados (use Ctrl+Espaço)

	AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1	id	int(11)	NO	PRI	[NULL]	auto_increm
2	nome	varchar(100)	NO		[NULL]	
3	email	varchar(150)	NO	UNI	[NULL]	
4	senha_hash	varchar(255)	NO		[NULL]	
5	criado_em	timestamp	YES		current_timestamp()	
6	role	enum('usuario','admin')	NO		usuario	

Figura 8: Atividade 2 – Tabela usuarios

COLUMNS 1 x

DESCRIBE comentarios | Insira uma expressão SQL para filtrar os resultados (use Ctrl+Espaço)

	AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1	id	int(11)	NO	PRI	[NULL]	auto_increm
2	usuario_id	int(11)	NO	MUL	[NULL]	
3	tmdb_movie_id	int(11)	NO		[NULL]	
4	texto	text	NO		[NULL]	
5	criado_em	timestamp	YES		current_timestamp()	

Figura 9: Atividade 2 – Tabela comentarios

DESCRIBE favoritos | Insira uma expressão SQL para filtrar os resultados (use Ctrl+Espaço)

	AZ Field	AZ Type	AZ Null	AZ Key	AZ Default	AZ Extra
1	id	int(11)	NO	PRI	[NULL]	auto_increm
2	usuario_id	int(11)	NO	MUL	[NULL]	
3	tmdb_movie_id	int(11)	NO		[NULL]	
4	titulo	varchar(255)	NO		[NULL]	
5	poster_path	varchar(255)	YES		[NULL]	
6	criado_em	timestamp	YES		current_timestamp()	

Figura 10: Atividade 2 – Tabela favoritos

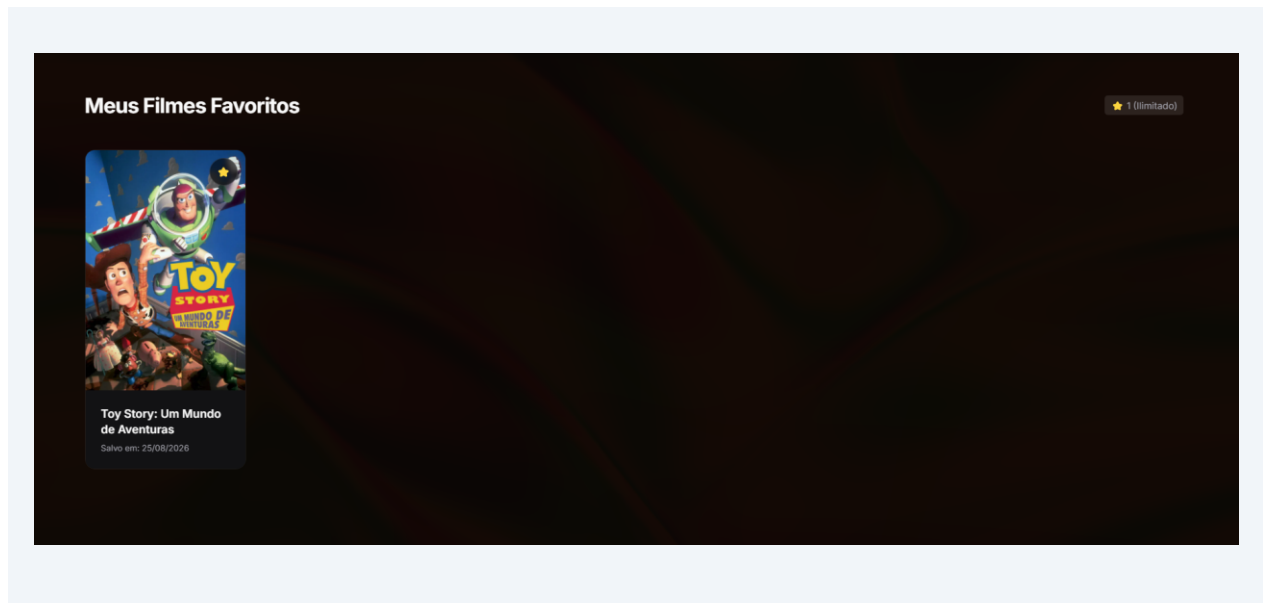


Figura 11: Atividade 2 – Favoritos do usuário

Dificuldades e como foram resolvidas. A API do TMDB utiliza paginação de resultados. Para garantir que toda a filmografia do ator fosse renderizada sem cortes, foi implementada uma rotina assíncrona que itera sobre as páginas da API e consolida os dados em memória antes da renderização EJS.

4.3 Atividade 3 – Desacoplando o login – microsserviço de autenticação

Descrição	Login, cadastro e esqueci-minha-senha num serviço à parte na rede interna do Docker.
Data planejada	28/08/2026
Data realizada	27/08/2026 13:00
Situação	entregue
Evidência	GitHub – commit 320028a + docker-compose.yml + print do login funcionando
Link	https://github.com/LucasHiratuca/Entregada-Atividade/commit/320028a

O que foi feito. A arquitetura do sistema foi refatorada sob o padrão Backend-for-Frontend (BFF). O serviço de catálogo (catalogo-service) foi mantido como único ponto de entrada público (porta 8217), enquanto toda a lógica de identidade, login, cadastro de novos usuários e redefinição de senha foi isolada no novo auth-service (porta interna 4000).

O auth-service não expõe portas ao host, comunicando-se exclusivamente pela rede interna Docker app_network. Ele emite tokens JWT criptograficamente assinados contendo a identidade do usuário. O fluxo de recuperação de senha foi integrado ao serviço SMTP do Mailtrap para despacho de e-mails com tokens descartáveis com expiração de 30 minutos.

```
320028a - término atividade 3 | 27/08/2026 às 13:00:45
6fcabdb - catalogo incompleto | 22/08/2026 às 14:19:06
```

Figura 12: Atividade 3 – Entrega do Commit

```
▷ Run Service
auth-service:
  build: ./auth-service
  image: lucashiratuca/auth-service:latest
  container_name: auth_service
  restart: always
  env_file:
    - .env
  environment:
    - LOG_SERVICE_URL=http://log-service:5000
  networks:
    - app_network
  depends_on:
    - log-service

▷ Run Service
log-service:
  build: ./log-service
  image: lucashiratuca/log-service:latest
  container_name: log_service
  restart: always
  env_file:
    - .env
  environment:
    - REDIS_HOST=redis
    - REDIS_PORT=6379
  networks:
    - app_network
  depends_on:
    - redis
```

Figura 13: Atividade 3 – Desacoplamento

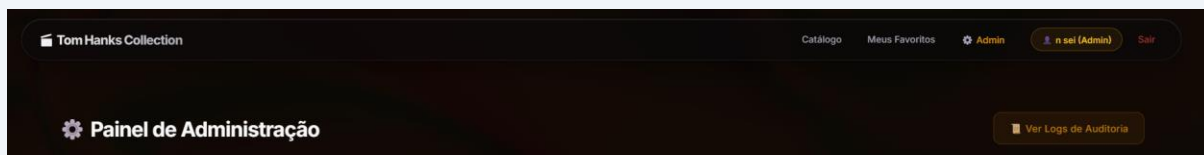


Figura 14: Atividade 3 – Admin

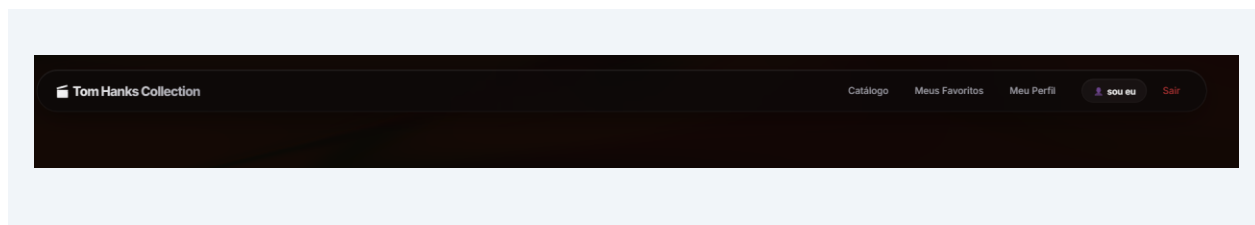


Figura 15: Atividade 3 – User

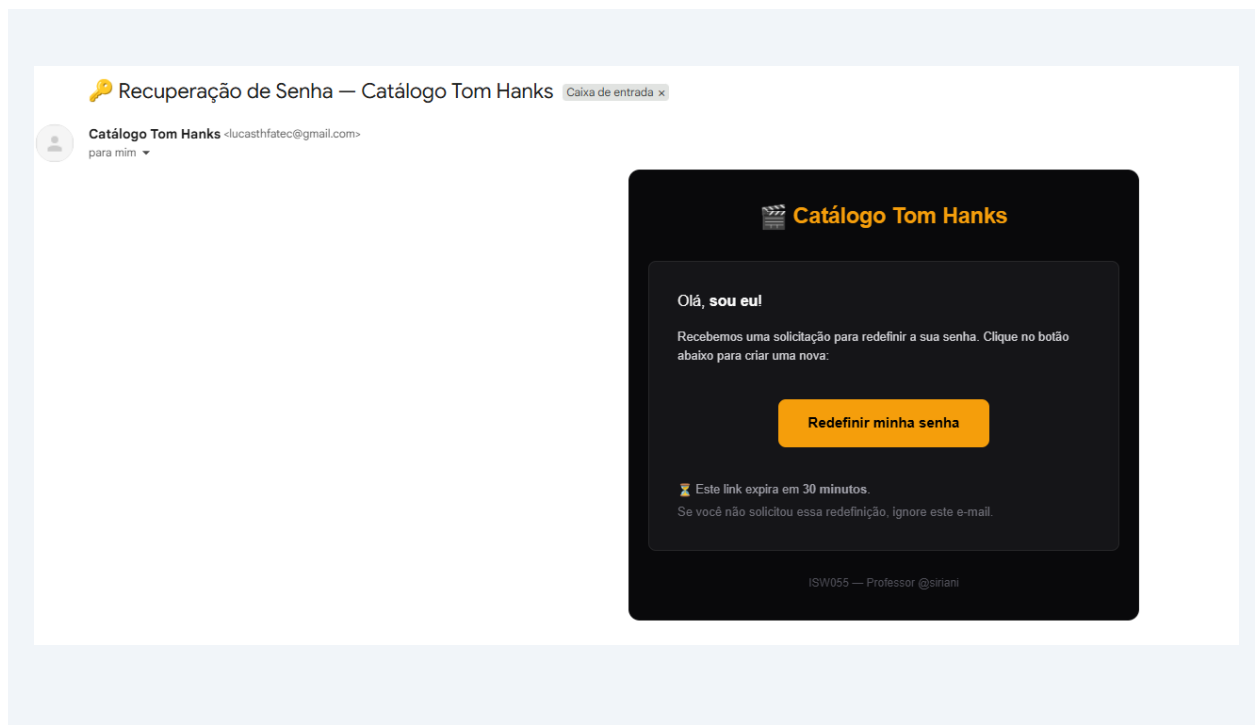
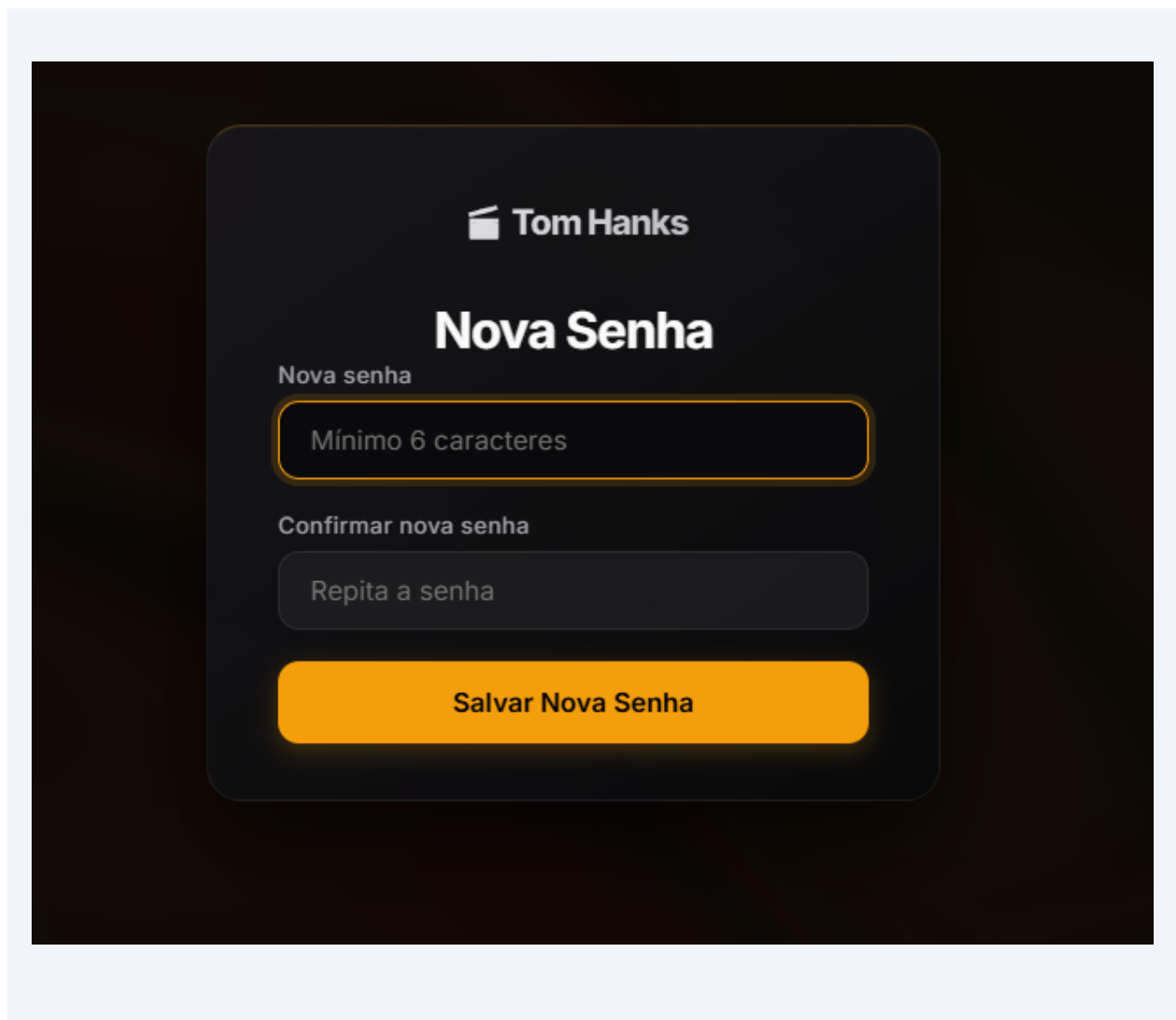


Figura 16: Atividade 3 – Email recuperar senha



The image shows a dark-themed user interface for a password reset. At the top, there is a logo consisting of a small square icon followed by the text "Tom Hanks". Below the logo, the title "Nova Senha" is displayed in a large, bold, white font. Underneath the title, the label "Nova senha" is shown in a smaller, lighter font. This is followed by a text input field with a yellow border and the placeholder text "Mínimo 6 caracteres". Below this field, the label "Confirmar nova senha" is shown in a smaller, lighter font, followed by another text input field with a dark gray border and the placeholder text "Repita a senha". At the bottom of the form, there is a prominent yellow button with the text "Salvar Nova Senha" in black.

Tom Hanks

Nova Senha

Nova senha

Mínimo 6 caracteres

Confirmar nova senha

Repita a senha

Salvar Nova Senha

Figura 17: Atividade 3 – Link de recuperar senha



Bem-vindo de volta

Acesse para explorar o catálogo exclusivo



Senha redefinida com sucesso! Faça login com a nova senha.

E-mail



seu@email.com

Senha



.....

Acessar Conta →

Esqueceu sua senha?

Não tem uma conta? **Cadastre-se grátis**

Figura 18: Atividade 3 – Senha recuperada

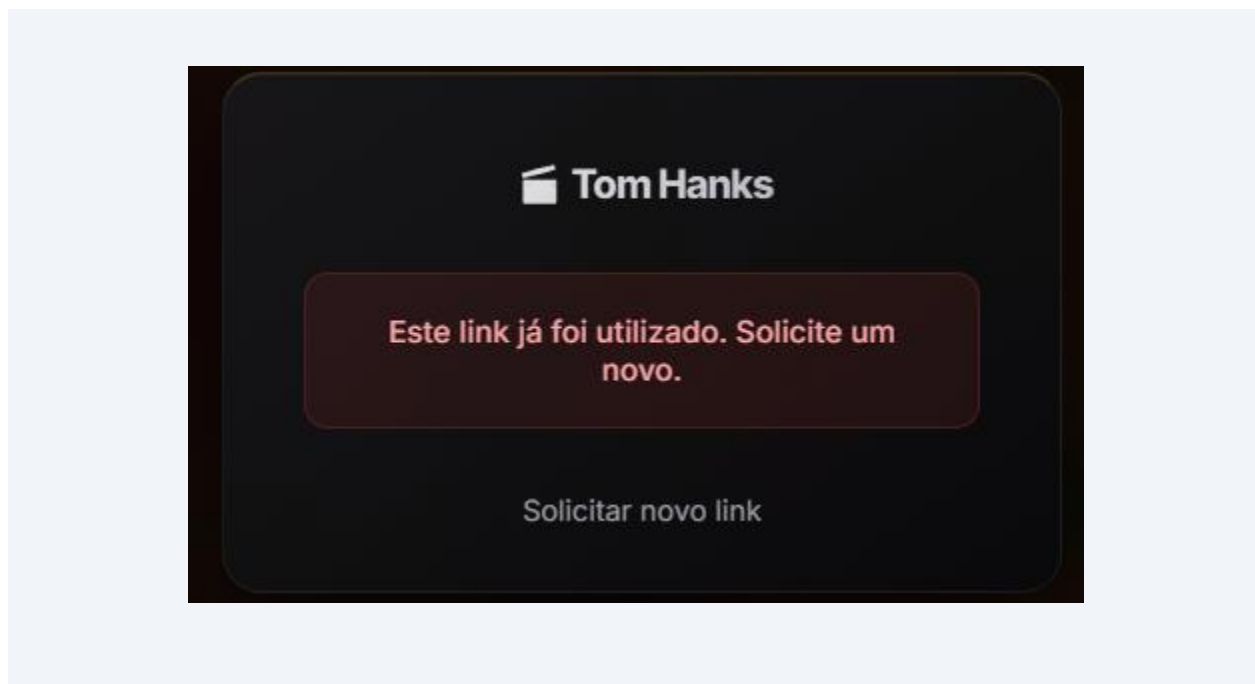


Figura 19: Atividade 3 – Linx expirado/usado

Dificuldades e como foram resolvidas. Ajustar a comunicação segura entre contêineres na mesma rede virtual do Docker Compose sem depender de IPs estáticos. A resolução foi alcançada utilizando os nomes dos serviços como hostnames DNS internos do Docker (<http://auth-service:4000>).

4.4 Atividade 4 – Controle de acesso por papel – RBAC

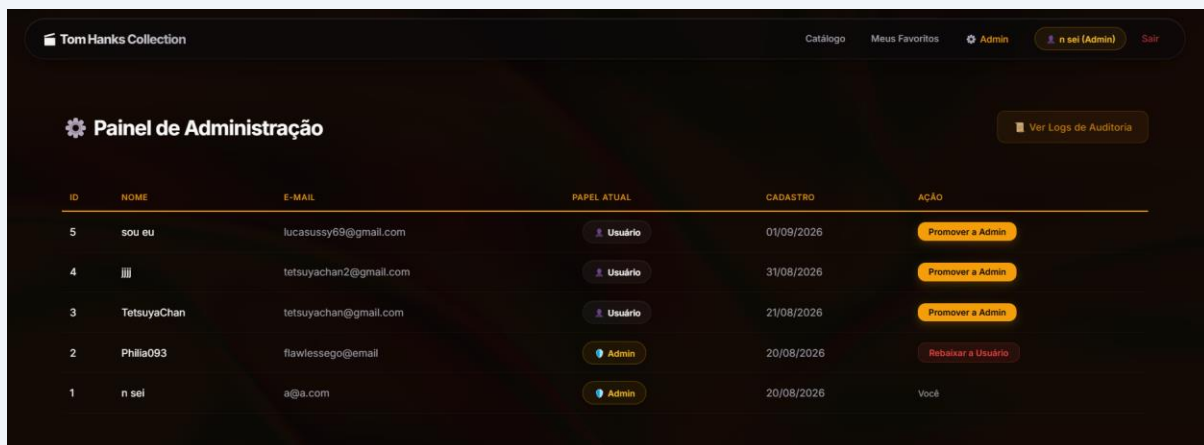
Descrição	O campo role passa a decidir permissões reais no backend (403 para usuário comum).
Data planejada	04/09/2026
Data realizada	01/09/2026 00:48
Situação	entregue
Evidência	GitHub – commits 6636f4d e a8e31ef + print do 403 e da ação de admin
Link	https://github.com/LucasHiratuca/Entregada-Atividade/commit/6636f4d

O que foi feito. Implementou-se o modelo de autorização Role-Based Access Control (RBAC) adotando o Padrão B (claims embutidas diretamente no payload do JWT). O campo role (usuario ou admin) trafega assinado no token, permitindo verificação instantânea no catalogo-service sem overhead de consultas repetidas ao banco de dados.

Foram criados os middlewares requireLogin e requireAdmin. Usuários autenticados com perfil comum têm acesso restrito ao catálogo, favoritos e comentários próprios. Tentativas de acessar rotas de moderação ou painel de gestão de usuários (/admin/usuarios) são barradas com status HTTP 403 Forbidden diretamente na camada do servidor.

```
a8e31ef - feat 5 final | 01/09/2026 às 21:36:42
6636f4d - refactor + feat | 01/09/2026 às 00:48:04
```

Figura 20 Atividade 4 – Evidência do Commit



The screenshot shows the 'Painel de Administração' (Admin Panel) of the 'Tom Hanks Collection' application. The panel includes a navigation bar with links to 'Catálogo', 'Meus Favoritos', 'Admin', and a user profile 'n sei (Admin)' with a 'Sair' (Logout) button. A 'Ver Logs de Auditoria' button is also present. The main content area displays a table of users with columns for ID, Nome, E-mail, Papel Atual, Cadastro, and Ação.

ID	NOME	E-MAIL	PAPEL ATUAL	CADASTRO	AÇÃO
5	sou eu	lucasussy69@gmail.com	Usuário	01/09/2026	Promover a Admin
4		tetsuyachan2@gmail.com	Usuário	31/08/2026	Promover a Admin
3	TetsuyaChan	tetsuyachan@gmail.com	Usuário	21/08/2026	Promover a Admin
2	Philia093	flawlessego@email	Admin	20/08/2026	Rebaixar a Usuário
1	n sei	a@a.com	Admin	20/08/2026	Você

Figura 21: Atividade 4 –Tela de admin (Exclusiva da role)

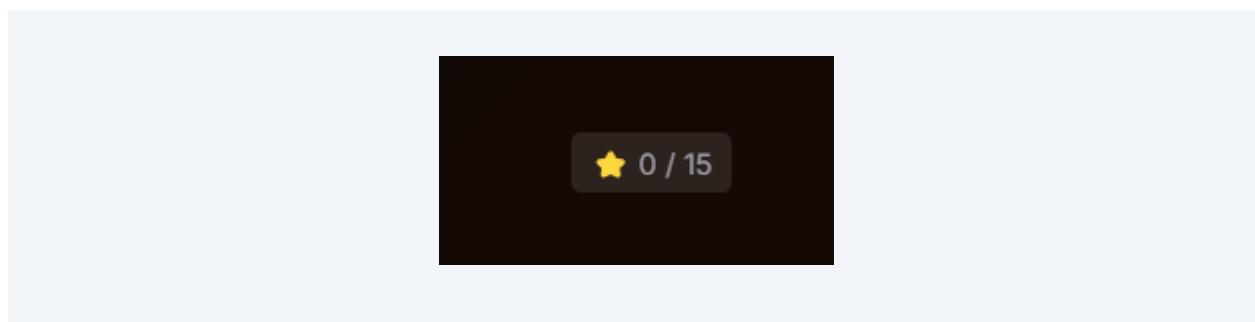
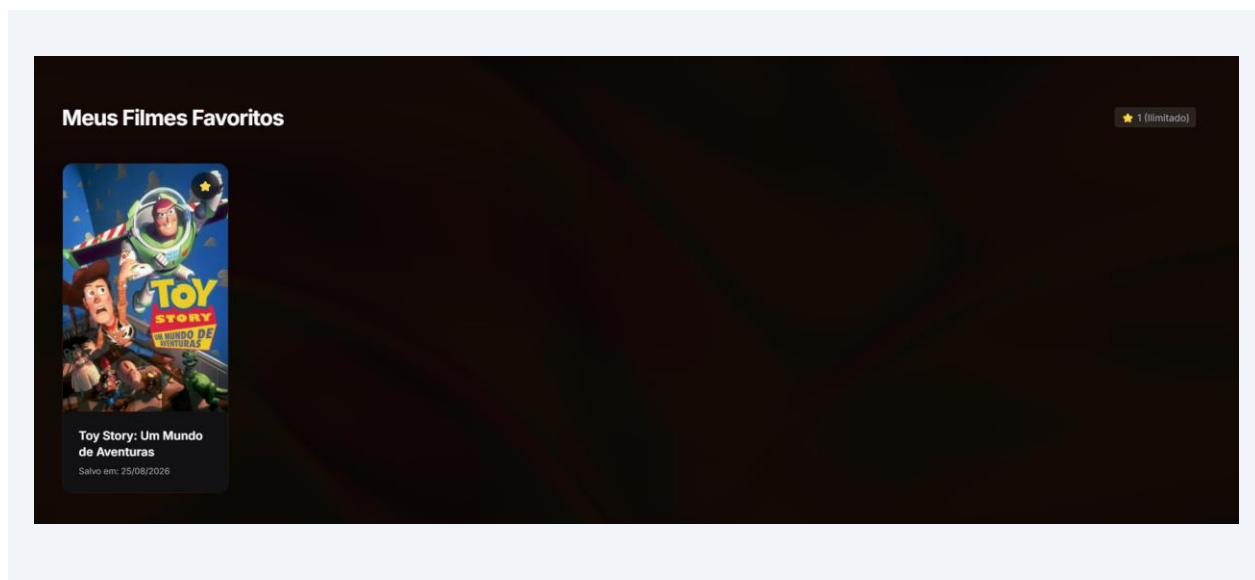


Figura 22-23: Atividade 4 – Favoritos ilimitados (Exclusiva Admin)

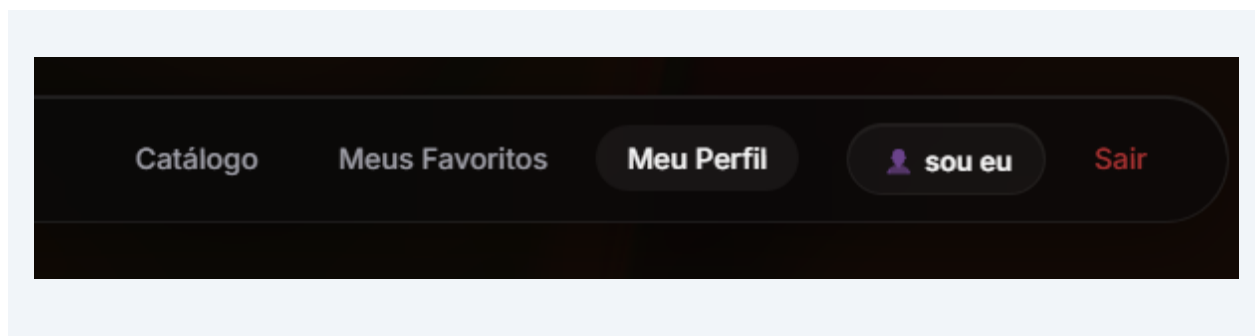


Figura 24: Atividade 4 –User não pode ver tela de admin

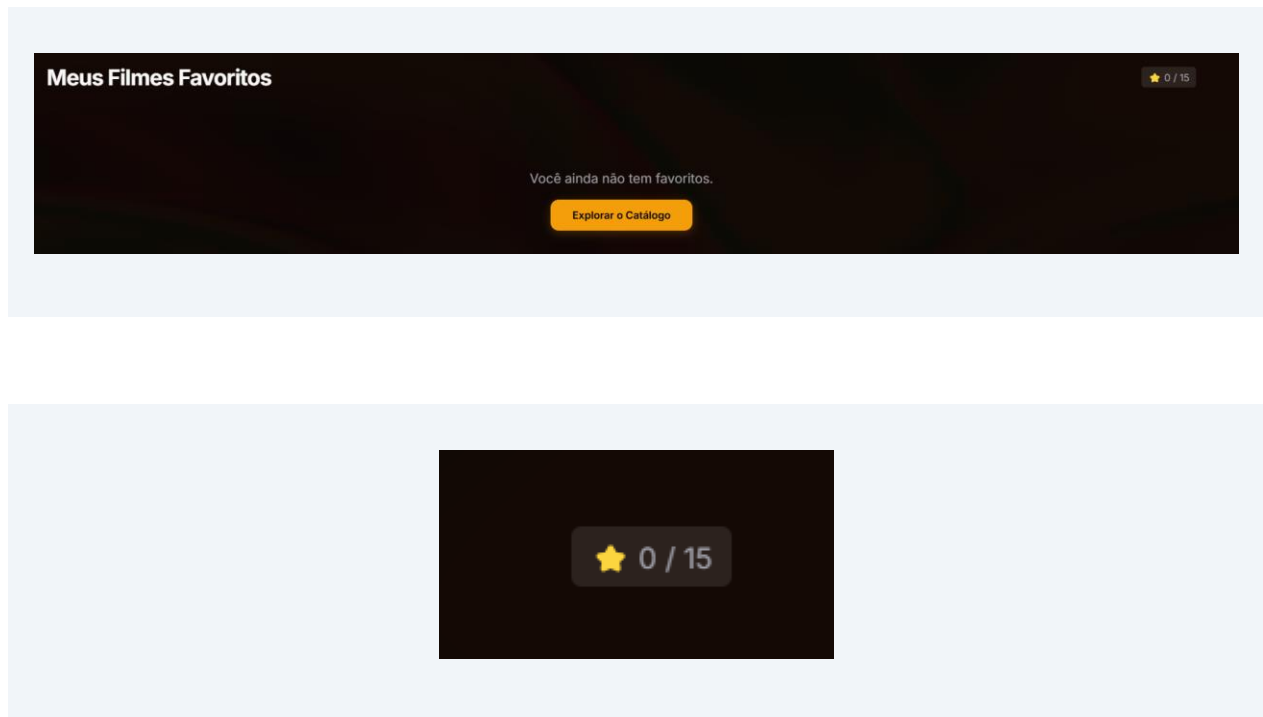


Figura 25-26: Atividade 4 – Limite de 15 favoritos em user

Dificuldades e como foram resolvidas. Garantir que tipos de dados inteiros para `userId` não causassem falhas na comparação estrita (`===`) no JavaScript. Foi adotado parsing consistente com `parseInt()` em todos os middlewares e controladores de permissão.

4.5 Atividade 5 – Logs e auditoria

Descrição	Novo log-service com Redis registrando login, ações sensíveis e tentativas negadas.
Data planejada	25/09/2026
Data realizada	08/09/2026 12:57
Situação	entregue
Evidência	GitHub – commits e6c6682 e 8bc9faa + print da consulta de logs pelo admin
Link	https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/e6c6682

O que foi feito. Foi introduzido um terceiro microserviço dedicado, o log-service (porta 5000), acoplado a uma instância do Redis 7. O serviço implementa ingestão assíncrona de eventos via HTTP POST, persistindo as entradas de auditoria na estrutura de alta performance Redis Streams.

O catalogo-service consome o utilitário de auditoria registrando eventos críticos: logins com sucesso, encerramentos de sessão, falhas de autenticação, exclusão de comentários e edições de perfil. A interface /admin/logs foi criada com restrição exclusiva a administradores para visualização e filtragem cronológica reversa dos registros em tempo real.

A tela de log tem acesso restrito para apenas users que tem cargo de Admin, por meio de um botão que aparece na tela exclusiva dos administradores. Como mostrado na entrega anterior, um usuário padrão não consegue ter acesso à tela especial dos admins

```
664cbb9 - fix no encoding + senha | 10/09/2026 às 15:14:42
ef488af - fix: bug na auth e log | 09/09/2026 às 00:38:28
e6c6682 - feat: logs | 08/09/2026 às 12:57:21
```

Figura 27: Atividade 5 – Evidência do Commit

Exibindo 22 de 22 eventos capturados (Redis Streams).

DATA/HORA	USUÁRIO	AÇÃO	DETALHES
03/10/2026, 04:09:12	n sei ID: 1	comentar	Comentou no filme 497: "buxa"
03/10/2026, 04:09:00	n sei ID: 1	favoritar_filme	Favoritou o filme "À Espera de um Milagre" (id: 497)
03/10/2026, 04:00:52	n sei ID: 1	login	Login bem-sucedido (role: admin)
03/10/2026, 04:00:32	sou eu ID: 5	logout	Usuário fez logout
03/10/2026, 03:57:12	sou eu ID: 5	login	Login bem-sucedido (role: usuario)
03/10/2026, 03:57:01	n sei ID: 1	logout	Usuário fez logout
03/10/2026, 03:54:15	n sei ID: 1	login	Login bem-sucedido (role: admin)
03/10/2026, 03:54:02	sou eu ID: 5	logout	Usuário fez logout

Figura 28: Atividade 5 – Logs (Apenas admins)

Dificuldades e como foram resolvidas. Garantir a sobrevivência dos logs a reinicializações de contêineres. Foi configurado um volume nomeado persistente para o contêiner do Redis no docker-compose.yml, além de timeout não-bloqueante no envio de logs para evitar impacto na experiência do usuário final caso o Redis ficasse temporariamente indisponível.

4.6 Atividade 6 – Upload e perfil de usuário

Descrição	Página de perfil com avatar no Garage S3; só a referência fica no banco relacional.
Data planejada	02/10/2026
Data realizada	26/09/2026 15:11
Situação	entregue
Evidência	GitHub – commits 4219f05, 27856a8 e 8ca3fb6 + print do perfil com foto + SECURITY_AUDIT.md
Link	https://github.com/LucasHiratuca/Entrega-da-Atividade/commit/27856a8

O que foi feito. Construção da página de perfil com suporte a upload de foto de avatar persistida em object storage. Foi integrado o Garage S3 (dxflrs/garage:v1.0.1), um engine leve e distribuído escrito em Rust compatível com AWS S3 API (Signature V4). A tabela perfis no MariaDB armazena apenas a biografia e a chave textual do objeto (foto_key), desacoplando o binário do banco relacional.

A entrega da imagem foi arquitetada via streaming seguro pelo backend (/perfil/:userId/foto): o catalogo-service obtém o stream do Garage S3 via rede interna e o devolve ao navegador com cabeçalhos HTTP adequados (Content-Type validado e Cache-Control). O upload exige validação de MIME type (JPEG/PNG/WebP/GIF), limite de 5MB e geração de nomes via randomUUID(). Foi realizada ainda auditoria OWASP Top 10 com 100% de aprovação e remediação do CWE-640 (Password Reset Poisoning).

```
27856a8 - feat: migracao de MinIO para Garage S3 e correcao na exibicao de foto | 26/09/2026 às 15:11:194c9e0 - fix: usar imagem cgr.dev/chainguard/minio:latest compativel | 26/09/2026 às 14:53:174219f05 - feat: pagina de perfil com upload de foto no MinIO | 26/09/2026 às 14:43:17
```

Figura 29: Atividade 6 – Evidência do Commit



Figura 30: Atividade 6 –Foto de perfil + bio

Dificuldades e como foram resolvidas. Em outubro de 2025, a imagem oficial do MinIO foi descontinuada do Docker Hub, motivando a migração definitiva para o Garage S3. Além disso, URLs pré-assinadas geravam links com o host interno do contêiner (garage:3900), inacessível aos navegadores dos clientes. A substituição por streaming direto no backend resolveu a compatibilidade universalmente em ambientes locais e Cloudflare.

5 Considerações finais

O percurso de desenvolvimento percorrido ao longo do primeiro bimestre de ISW055 permitiu consolidar na prática os fundamentos de arquiteturas em nuvem modernas. A transição de um script inicial monolítico em Python para um ecossistema completo composto por microsserviços containerizados em Node.js, mensageria com Redis e storage compatível com S3 proporcionou um aprendizado expressivo sobre segregação de responsabilidades e resiliência.

A maior complexidade encontrada ocorreu na Atividade 6, onde a descontinuidade do MinIO no Docker Hub exigiu adaptabilidade imediata para selecionar, configurar e implantar o Garage S3 em Rust, ajustando o layout de cluster e o roteamento interno de rede. Essa experiência evidenciou o valor de adotar padrões abertos de mercado (como a S3 API), permitindo trocar a engine de armazenamento sem alterar as rotas da aplicação.

Como aprendizado contínuo, destaca-se a relevância de conceber sistemas com foco em segurança desde a camada de design (Security by Design). A auditoria baseada no OWASP Top 10 permitiu antecipar vetores de ataque como enumeração de credenciais e envenenamento de links. Para o segundo bimestre, pretende-se aprofundar na instrumentação de métricas com Prometheus e automação de pipelines de entrega contínua (CI/CD).

6 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. Nas atividades realizadas em grupo, o conteúdo aqui descrito refere-se à minha participação.

Lucas Hiratuca

Aluno(a)

Pompeia, 07/10/2026